# Advanced Software Metrics Analysis Dashboard

for

# Code Analysis

**Version 1.0 approved by Muhammad Farooq**

**Prepared by Ahmad Hassan**

**Air University**

**12 Dec 2024**

Copyright © 2024 by Ahmad Hassan 221775

## Table of Contents


..... 1

Table of Contents ..... 2

Final Assignment Report: Analysis and Recommendations on Software Metrics ..... 3

I. Methodology for Selecting and Calculating Metrics ..... 3

1. Code Metrics ..... 3

2. Agile Metrics ..... 5

3. COCOMO Effort Estimation ..... 6

4. Visualizations ..... 7

II. Influence of Metrics on Software Development Lifecycle ..... 10

1. Code Quality and Maintainability ..... 10

2. Resource Allocation and Effort Estimation ..... 10

3. Agile Practices ..... 11

4. Risk Management ..... 12

III. Suggested Improvements Based on Metrics ..... 13

1. Code Quality Enhancements ..... 13

2. Testing Focus ..... 13

3. Agile Practice Refinements ..... 13

4. Resource and Timeline Adjustments ..... 14

5. Continuous Monitoring ..... 14

Conclusion ..... 16


Copyright © 2024 by Ahmad Hassan 221775

# Final Assignment Report: Analysis and Recommendations on Software Metrics

## I. Methodology for Selecting and Calculating Metrics

To analyze and improve software development practices, it is crucial to assess various aspects of the development process. A set of carefully chosen metrics was used to evaluate critical areas such as code quality, project complexity, team performance, and overall management efficiency. These metrics were categorized into different groups to allow for a detailed, structured analysis.

### 1. Code Metrics

- **Selection Process:** The code-related metrics were selected to capture key aspects that directly impact the maintainability and reliability of the codebase. Metrics like Lines of Code (LOC), Cyclomatic Complexity, Cognitive Complexity, Function Points, and Defect Density were chosen for their strong correlation to code quality and testability.
    - **Lines of Code (LOC)**: Measures the size of the codebase, providing a basic understanding of the amount of code that needs to be maintained and tested. A large codebase often indicates a higher level of complexity and more room for errors.

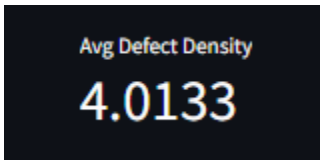
Copyright © 2024 by Ahmad Hassan 221775

- **Cyclomatic Complexity**: Assesses the logical complexity of the code by counting the number of linearly independent paths through the program. It helps identify areas of code that are difficult to understand and test.

- **Cognitive Complexity**: Unlike Cyclomatic Complexity, this metric evaluates the difficulty in understanding the code. It accounts for control flow constructs like loops and conditionals, providing a clearer picture of how easy or hard it is to comprehend a module or function.

- **Function Points**: Provides an estimate of the functionalities delivered by the system, which is crucial for understanding the scope of the project and evaluating the effort needed for development.

- **Defect Density**: By measuring the number of defects per unit of code, this metric helps in identifying areas of the code that are prone to errors and require more thorough testing.

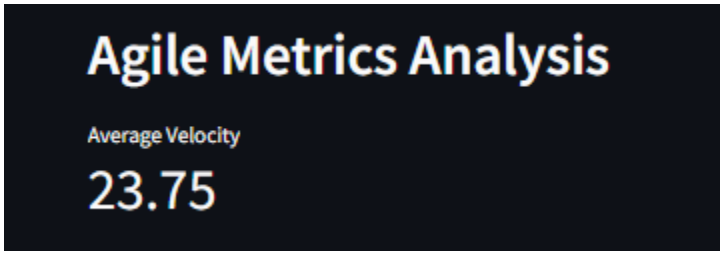



Copyright © 2024 by Ahmad Hassan 221775

- **Calculation Method:**
    - Python's Abstract Syntax Tree (AST) was employed to parse the source code for extracting and analyzing metrics such as LOC and complexity scores.
    - Defect density was calculated by applying a weighted function that combined complexity and LOC, identifying areas of the code that required more attention from both developers and testers.

## 2. Agile Metrics

- **Selection Process:** Agile metrics were selected to monitor team productivity and project progress. These include metrics like Velocity and Scope Creep, which are particularly useful in understanding how effectively a team is working within an Agile framework.
    - **Velocity**: Tracks the average amount of work completed by the team per sprint, helping to forecast future sprint capacities and identify any potential bottlenecks.




    - **Scope Creep**: Monitors unplanned work added during a sprint, which can disrupt timelines and lead to inefficiencies. It helps in managing expectations and controlling the scope of work throughout the project.

Copyright © 2024 by Ahmad Hassan 221775

- **Calculation Method:**
    - Agile metrics were derived by analyzing JSON data from multiple sprints. The velocity was calculated by averaging the story points or tasks completed per sprint.
    - Scope creep was measured by identifying deviations from the original sprint backlog and quantifying the unplanned work added during the sprint cycle.

## 3. COCOMO Effort Estimation

- **Selection Process:** To estimate the effort and resources required for the project, the Constructive Cost Model (COCOMO) was applied. This model is particularly useful in predicting the amount of effort (in person-months) and time (in months) needed to complete a software project based on its size (measured in thousands of lines of code, or kLOC).

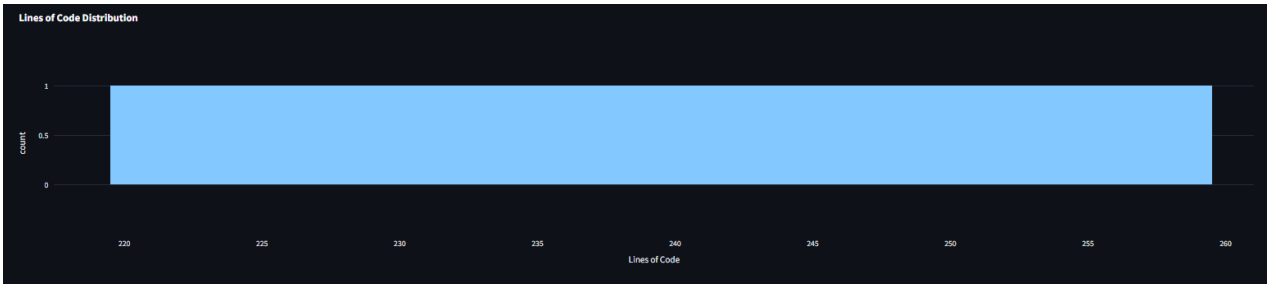
- **Calculation Method:**

Copyright © 2024 by Ahmad Hassan 221775

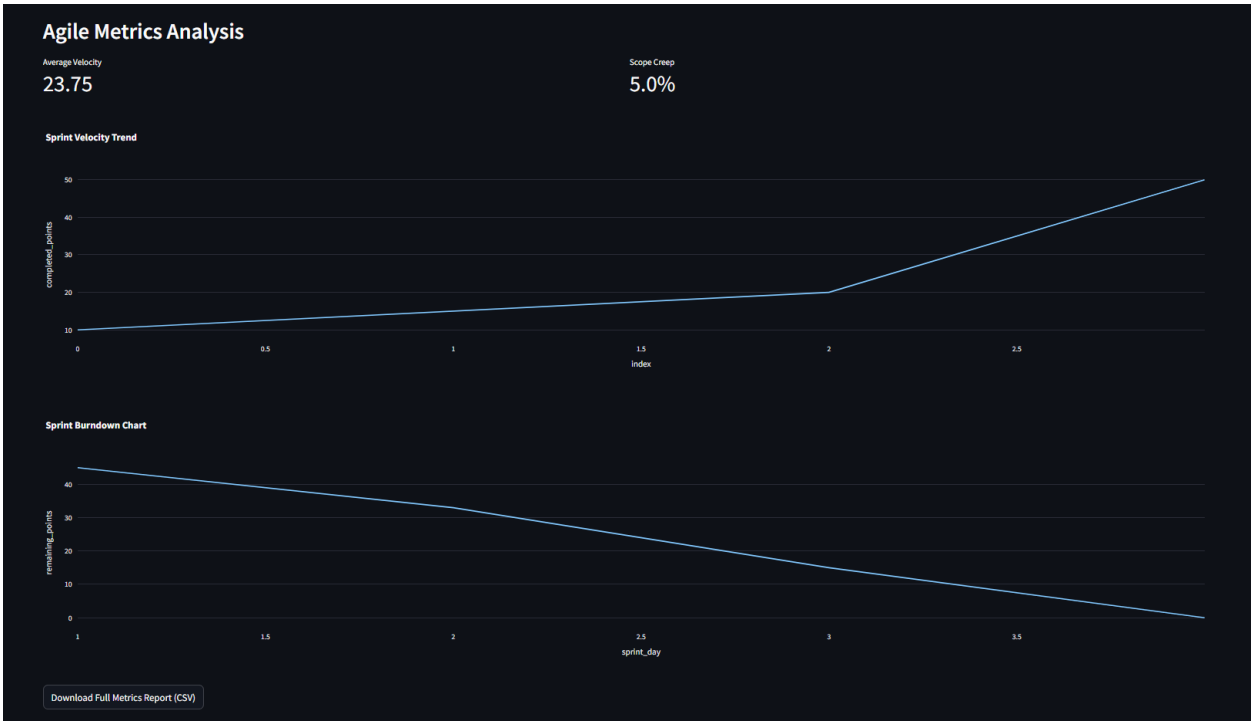
- The organic model of COCOMO was used, which assumes the project is relatively simple and well-understood. This model provided projections for effort, timelines, and staffing needs based on the size of the codebase.
- The results helped in determining whether the resources allocated for the project were adequate or needed adjustment.

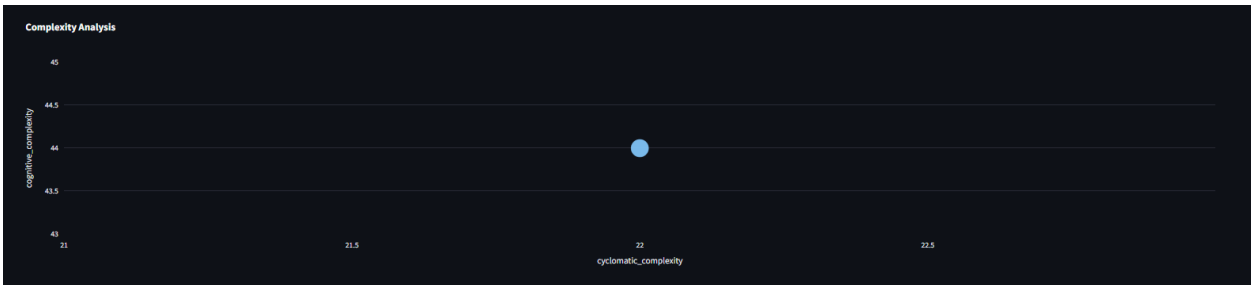
## 4. Visualizations

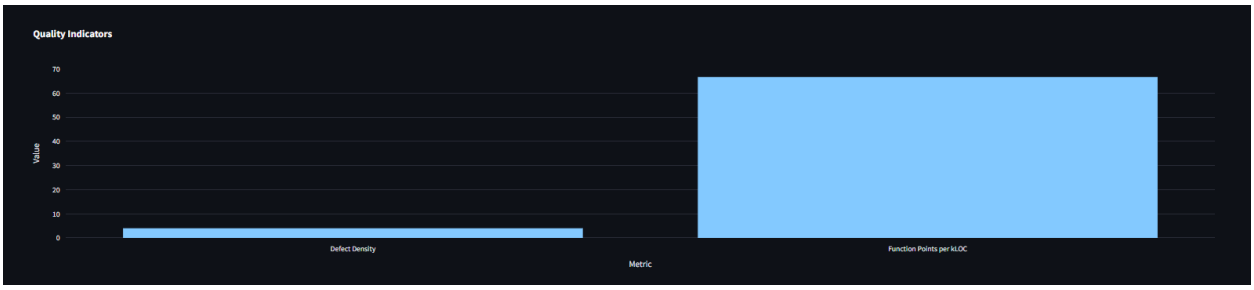
- **Selection Process:** Various charts and graphs were chosen to effectively communicate the findings from the metrics. Histograms, scatter plots, and burndown/burnup graphs were used to visualize code distributions, detect outliers in complexity, and track sprint progress.




Copyright © 2024 by Ahmad Hassan 221775




Copyright © 2024 by Ahmad Hassan 221775

- **Tools Used:**
    - Visualization tools like Plotly were employed to create interactive, dynamic charts, making it easier to interpret the data and gain insights from the metrics.
    - These visualizations provided both technical and non-technical stakeholders with a clear understanding of the project's progress and health.

Copyright © 2024 by Ahmad Hassan 221775

## II. Influence of Metrics on Software Development Lifecycle

### 1. Code Quality and Maintainability

- **Impact:** The chosen metrics had a significant impact on understanding code quality:
  - High cyclomatic and cognitive complexity values indicated areas where code could be simplified, improving readability and reducing the likelihood of errors.
  - Function points clarified the scope of deliverables, making it easier to align developer efforts with project goals.
  - Defect density helped identify high-risk areas in the code that needed more rigorous testing and quality assurance processes.
- **Decisions Affected:**
  - These insights guided decisions regarding refactoring efforts and prioritizing code areas for testing.
  - Teams focused on simplifying complex modules to improve overall maintainability and reduce future technical debt.

### 2. Resource Allocation and Effort Estimation

- **Impact:** The COCOMO model provided crucial estimates regarding the effort and timeline required to complete the project. These projections enabled more accurate resource allocation, ensuring that the development team had the right number of staff for the right tasks.

Copyright © 2024 by Ahmad Hassan 221775

- By integrating LOC and function point data, more precise scope definitions were created, helping to prevent scope creep and manage project timelines effectively.

- **Decisions Affected:**
  - With these estimations in hand, staffing adjustments were made, and project timelines were adjusted to align with the predicted effort required.
  - The model also helped phase tasks more effectively to ensure milestones were met in a timely manner.

## 3. Agile Practices

- **Impact:**
  - Agile metrics such as velocity provided insights into how the team was progressing with their workload and helped forecast future sprint capacities.
  - Monitoring scope creep during sprints revealed inefficiencies, indicating areas where planning or execution could be improved.

- **Decisions Affected:**
  - The team was able to refine sprint planning, ensuring that the scope of work remained manageable.
  - This led to more accurate estimations of what could be accomplished in each sprint and improved the overall efficiency of the Agile process.

Copyright © 2024 by Ahmad Hassan 221775

## 4. Risk Management

- **Impact:**
  - Monitoring metrics such as complexity and scope creep allowed the team to identify and address potential risks early in the development process.
  - Complex code modules were flagged for refactoring, reducing the long-term risk of defects and maintenance challenges.
  - Similarly, tracking scope creep helped avoid project delays caused by uncontrolled changes in the sprint scope.
- **Decisions Affected:**
  - Proactive risk management led to the implementation of corrective actions, such as resource reallocation or changes to the sprint backlog, ensuring that risks were mitigated before they could affect project delivery.

Copyright © 2024 by Ahmad Hassan 221775

## III. Suggested Improvements Based on Metrics

### 1. Code Quality Enhancements

- **Recommendations:**
  - Refactor complex modules into smaller, more manageable components.
  - Standardize coding practices to reduce cognitive load and simplify the codebase.
  - Encourage the use of design patterns to ensure maintainable and scalable solutions.

### 2. Testing Focus

- **Recommendations:**
  - Prioritize testing in areas with high defect density and high cyclomatic or cognitive complexity.
  - Incorporate automated testing for complex modules to enhance coverage.
  - Establish a comprehensive code review process to catch defects early in the development process.

### 3. Agile Practice Refinements

- **Recommendations:**
  - Reduce scope creep by refining sprint planning and ensuring clear scope definition at the start of each sprint.

Copyright © 2024 by Ahmad Hassan 221775

- Limit mid-sprint additions and refine requirement gathering processes to prevent last-minute changes.
- Introduce stricter rules for scope changes and additional work during sprints to maintain focus and deliver on time.

## 4. Resource and Timeline Adjustments

- **Recommendations:**
  - Allocate resources strategically to address high-complexity modules.
  - Realign timelines based on COCOMO's effort predictions, ensuring the project schedule reflects the real effort required.
  - Reassess team roles and assign resources based on expertise, ensuring critical tasks are completed on time.

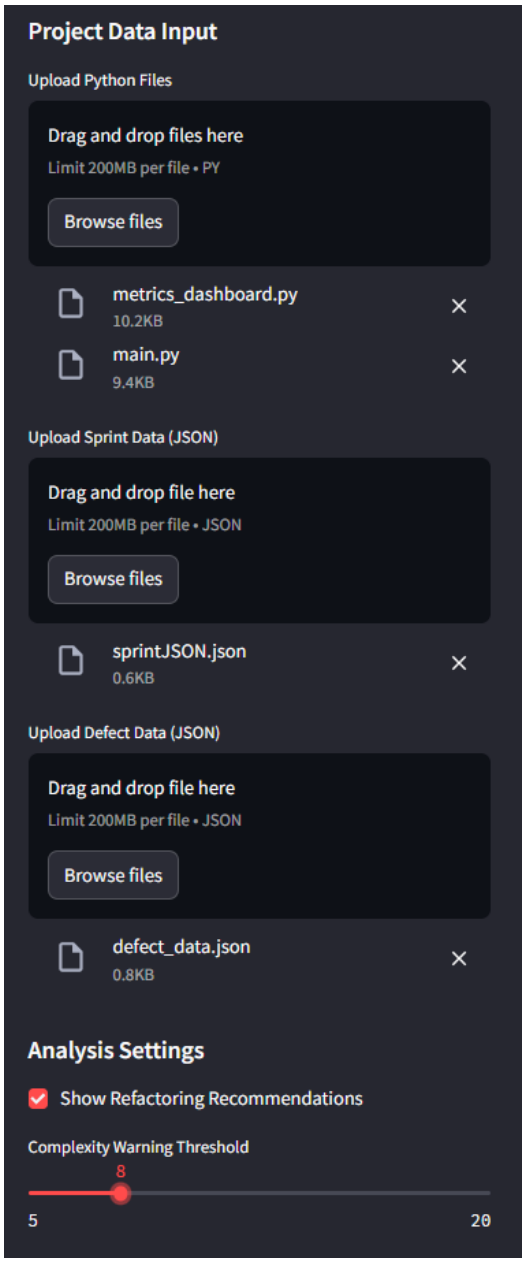
## 5. Continuous Monitoring

- **Recommendations:**
  - Integrate metric tracking into CI/CD pipelines for continuous monitoring of code quality and Agile progress.
  - Use interactive dashboards for real-time monitoring of key performance indicators (KPIs), allowing quick adjustments as needed.
  - Set up automated alerts to notify teams when certain thresholds for complexity, defects, or velocity are breached.

Copyright © 2024 by Ahmad Hassan 221775

## 6. Sample Output:

## File Adders:




Copyright © 2024 by Ahmad Hassan 221775

## Conclusion

The analysis of software metrics revealed valuable insights into various aspects of the software development lifecycle, from code quality and complexity to team performance and risk management. By implementing the suggested improvements, teams can enhance code maintainability, reduce defects, and optimize Agile processes. These changes will ultimately lead to more successful project outcomes, with greater efficiency, fewer errors, and a more predictable development timeline

Copyright © 2024 by Ahmad Hassan 221775